整数类型整体分两类： 无符号数、有符号数
一定要注意，C标准并没有规定int一定占4个字节。 所以大家一定要注意这里潜藏的问题
要大致知道一下范围：
short(2) 3w多。 如果超过了，不要用short
int(4) -21亿，+21亿
平常 99%都是使用int。 int装不下，用其他的。
如果我们要明确指出一个字面值为long类型。 在后面加，但是禁止加 小写l，一律用大写的L。
10l 101
long num = 20L;

浮点数不准确。

科学计数法。 1000 1.0*10^3

字符型就是用整形来存的。
编码表是什么东西?
就是一个对应关系，比如用97来存储a 98来存储b。 ASCII是美国国家标准委员会制定的一个对应关系。GBK， UTF-8

C语言的基本数据类型有且只有两类：
1.整型：char、short、int、long、long long 这些整型都具有有符号和无符号两种形态
2.浮点型：float、double、long double
特别需要注意，C语言没有布尔的基本数据类型。
C语言把类型（基本数据类型和指针类型）的零值看成是布尔值false，非0值看成true
零值：整型就是0，浮点型就是0.0，指针类型是NULL（空指针）
注意：
关于在if判断当中使用布尔表达式的问题
if(x == 10){
...
}
如果写成：
if(x = 10){
...
}
这样的话，if始终为true，始终执行...不要误把"==", 写成"="
if(p != NULL) 等价于 if(p)，我更倾向于写前面的，可读性会更好

关于C语言的布尔类型，更简易包含<stdbool.h>头文件，然后使用bool类型明确表示布尔类型，以及true和false来表示真和假。

整型可以展开讲一下，整型可以再分为两类：
1.short、int、long、long long 是一类，它们是纯粹的整数，纯粹的整型。
C语言的基本数据类型的长度大多是不确定的，以上四种类型，C语言标准只规定了它们长度的大小关系以及最小长度
short(2)、 int(2)、long(4)、long long(8)
short <= int <= long <= long long
这里面最需要注意的是int类型，在大多数普通机器上，int类型都是4个字节，但是在某些小型机或嵌入式开发中，int常常是2个字节
C语言代码中，整型字面值常量的表示形式：
1.如果以0x开头，表示十六进制整数
2.如果以0开头，表示八进制整数
3.以上都不是，就是十进制整数
注意C语言不支持，直接写二进制整数。

2.char类型（注意）
char类型既可以表示字符类型，也可以表示小的整数类型。
在表示字符类型时，char类型就是ASCII码表中字符的编码值。
在表示整数时，char被视为1个字节的小整型。
（重点）char的长度是C标准明确规定的，一定是1个字节。但是C标准没有规定char类型的有无符号，有可能是有符号，也有可能是无符号的
在MSVC平台下，char是有符号的

sizeof运算符，用来测量，计算，某个类型、变量的长度（所占内存空间的大小），以字节为单位，而且结果一定是非负数，实际上也不会是0
// sizeof重要的惯用法: 用于定义宏函数,求数组长度
#define ARR_SIZE(arr) (sizeof(arr) / sizeof(arr[0]))
sizeof(arr) sizeof连接数组名, 在正常情况下(挖坑), 表示求整个数组所占内存空间的字节数量
然后除以一个存储单元所占内存空间的字节数量, 就是数组的长度
sizeof(arr[0])
C语言不允许空数组, 数组长度至少也是1, 至少也有1个元素
在C语言代码当中, 若需要使用类型字节长度的数据, 建议不要直接写固定的整数, 而是使用sizeof运算符计算得出

回车和换行有什么区别?
回车和换行实际上是不同的,我们现在谈到的回车,其实就是换行
但是在早期, 在使用打字机的时代, 在输入数据时, 有一个真正的物理硬件叫做"车针"(类似在屏幕上输入时的竖杠)
打字员在实现换行时, 需要把车针回到行的开头(回车), 然后再把车针移动到下一行, 从而实现换行的操作
基于这样的特点, 在不同的操作系统中, 换行的实现是不同的(重点):
1.Windows平台, 继承了打字机的换行, 需要先回车再换行, 实现换行. 所以在Windows平台, 换行是 \r\n 两个字符组成的(万恶之源)
2.类Unix平台, 包括unix, Linux, MacOS, 它们摒弃了打字机的设计, 直接使用 \n 来表示换行符
由于换行符在这两个平台之间的差异性, 非常容易引发跨平台错误, 程序员需要重视

end of file = EOF
getchar函数在读stdin缓冲区时, 若读完了, 就会返回EOF
EOF在绝大多数平台上, 它实际上就是-1
getchar的返回值类型是int, 这和EOF是有关系的, 因为char可能是无符号的, 这样char就不能表示-1了
所以用int做返回值类型

类型别名的作用:
1.提升代码可读性. 别名往往比本名更好的描述类型的作用
举例:
链表是由结点链接构成的数据结构, 在C语言中, 链表的结点就是一个Node结构体类型,这个结构体就可以直接命名为Node类型
栈可以基于链表实现, 这种栈就是链式栈
链式栈肯定也是一个一个结点链接而成, 但是这个结点就不是普通的结点了, 而是栈的一个基本单元
于是我们就起别名: StackFrame

2.提升代码的扩展性
typedef int DataType;
DataType是链表中存储数据的类型, 在链表实现的所有位置都写DataType类型
这样设计后, 只需要修改类型别名的语句, 其余位置完全不改, 就可以实现链表存储其他类型

3.提升代码的跨平台性(重要)
typedef int BigNum;
在int是4个字节的平台上, 可以直接用int
在移植到int是2个字节的平台时,只需要改类型别名的语句为:
typedef long long BigNum;

在C语言的标准库当中, 提供了一些特别常用,有用的类型别名:
size_t: 在任何平台下, 它都表示和平台相关的无符号整数. 32位平台, 它就是32位无符号整数, 64位平台, 它就是64位无符号整数
该类型别名非常常见, 在标准库的函数中, 若函数明确返回非负数, 普遍都会用该类型别名作为返回值
int8_t, int16_t, int32_t, int64_t: 无论任何平台下，分别表示确切的8、16、32、64位有符号整数。
uint8_t, uint16_t, uint32_t, uint64_t: 无论任何平台下，分别表示确切的8、16、32、64位无符号整数。

无符号数只建议在以下场景使用:
数组的长度
字符串的长度
某个数据结构中的元素、结点的数量
内存大小，比如sizeof运算符的结果类型就是size_t